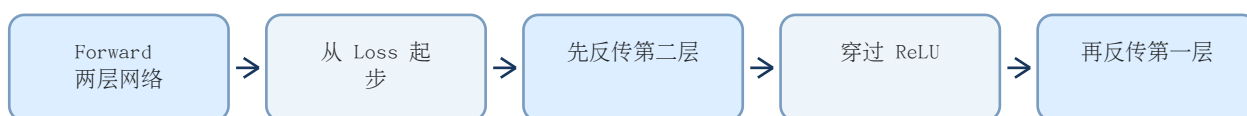


第25讲：两层 MLP 的 Backpropagation

从矩阵 Forward 到 $dW2$ 、 $db2$ 、 $dW1$ 、 $db1$

核心概念：【必须真正掌握】

本讲把上一讲的单神经元 Backpropagation 升级为真正的两层 MLP。目标是看懂每一层的 Tensor、Gradient、Transpose 和 Shape 怎样配合，让最终 Loss 一层一层传回 $W2$ 、 $b2$ 、 $W1$ 、 $b1$ ，最终还能继续传给输入。



建议学习方式：先跟着固定的 3->2->1 数字例子手算，再理解矩阵公式，最后运行 Numerical Gradient Check 与 PyTorch Autograd 对照。

一、本讲学习目标

看懂一个 3→2→1 两层 MLP 的完整 Forward 与 Backward。

理解为什么第二层 Weight 梯度写成 $dW2 = a1^T @ dy_hat$ 。

理解为什么传回隐藏层的梯度写成 $da1 = dy_hat @ W2^T$ 。

理解 ReLU mask 怎样把 $da1$ 变成 $dz1$ 。

理解为什么第一层 Weight 梯度写成 $dW1 = x^T @ dz1$ 。

理解为什么 $dx = dz1 @ W1^T$ ，从而梯度还能继续传到更前面的层。

形成“参数 Shape 与其 Gradient Shape 完全对应”的稳定习惯。

用手算、NumPy、Numerical Gradient Check、PyTorch Autograd 四种方式交叉验证。

为第26讲 NumPy 手写完整训练循环做好准备。

核心概念：与第24讲的关系

第24讲只有一个神经元，所以 Weight 是向量；第25讲同时有多个隐藏神经元，Weight 变成矩阵，Backward 也因此正式进入矩阵形式。

二、【原理】两层 MLP 到底是哪两层？

本讲使用最小但足够真实的网络：

$x \rightarrow \text{Linear1} \rightarrow \text{ReLU} \rightarrow \text{Linear2} \rightarrow y_{\text{hat}} \rightarrow \text{Loss}$

具体写成公式：

$$z1 = x @ W1 + b1$$

$$a1 = \text{ReLU}(z1)$$

$$y_{\text{hat}} = a1 @ W2 + b2$$

$$L = (y_{\text{hat}} - \text{target})^2$$

这里“2层”通常指两层可学习的 Linear：第一层把输入映射到 hidden space，第二层把 hidden representation 映射到输出。

科研连接：为什么它已经是真正的 MLP

因为两层 Linear 中间插入了非线性 ReLU，所以不能整体折叠成一层 Linear。第8讲学过的结论在这里正式发挥作用。

三、【Shape 地图】先不算数字，只看维度

网络结构是：输入维 3，隐藏维 2，输出维 1。

变量	Shape	作用
x	(1,3)	1 个样本，3 个输入特征
W1	(3,2)	3 维输入 -> 2 个隐藏神经元
b1	(2,)	2 个隐藏神经元各一个 Bias
z1 / a1	(1,2)	2 个 hidden values
W2	(2,1)	2 维隐藏表示 -> 1 个输出
b2	(1,)	输出层 Bias
y_hat / target	(1,1)	预测与目标
L	scalar	最终 Loss

核心概念：先看 Shape 再看公式

以后写任何 Backprop 推导，先把 Shape 写出来。绝大多数矩阵转置和乘法方向都能靠 Shape 自检，而不是靠死记。

四、【具体数字例子】完整 Forward

$$x = [1, 2, -1]$$

第一层参数:

$$W1 = [[0.5, -1], [1, 0.5], [-0.5, 2]], b1 = [0, 0]$$

先算:

$$z1 = x @ W1 + b1 = [3, -2]$$

$$a1 = \text{ReLU}(z1) = [3, 0]$$

第二层参数:

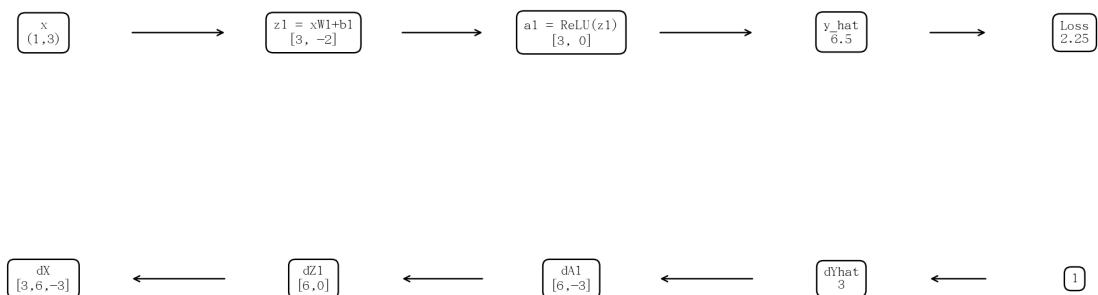
$$W2 = [[2], [-1]], b2 = [0.5]$$

$$y_hat = [3, 0] @ [[2], [-1]] + 0.5 = 6.5$$

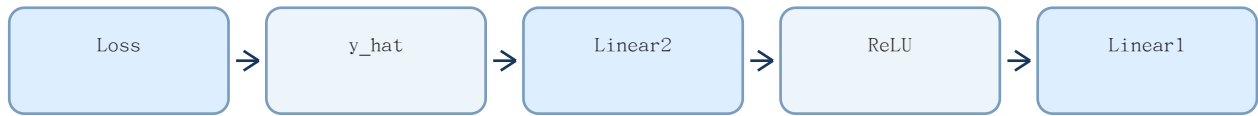
$$\text{target} = 5$$

$$\text{error} = 1.5, L = 1.5^2 = 2.25$$

两层 MLP: 上方 Forward, 下方 Backward



五、【Backward 总览】从 Loss 往回走的顺序



Backward 顺序与 Forward 相反：

步骤	要算什么	目的
1	dy_hat	Loss 对最终预测的敏感度
2	dW2, db2	第二层参数梯度
3	da1	把梯度传回 hidden activation
4	dz1	穿过 ReLU gate
5	dW1, db1	第一层参数梯度
6	dx	继续传给更前面的网络

核心概念：本讲真正的主线

每一层都做同样三件事：算本层参数梯度；算要传给前一层的梯度；保证所有 Shape 合法。

六、【Backward 1】先从 Loss 得到 dy_hat

$$L = (y_hat - target)^2$$

因此:

$$dL/dy_hat = 2(y_hat - target)$$

本例:

$$dy_hat = 2(6.5 - 5) = 3$$

数学直觉: 上游梯度

接下来整个第二层都把 $dy_hat=3$ 当成来自后方的“误差信号”。它会同时用于计算 $w2$ 、 $b2$ 的梯度, 也用于继续传回 $a1$ 。

七、【Backward 2】第二层 Weight 为什么是 $a1^T @ dy_hat$?

第二层:

$$y_hat = a1 @ W2 + b2$$

如果逐元素看:

$$y_hat = a1_1*w2_1 + a1_2*w2_2 + b2$$

因此:

$$dL/dw2_i = (dL/dy_hat) * a1_i$$

本例 $a1=[3,0]$, $dy_hat=3$:

$$dW2 = [[3*3],[0*3]] = [[9],[0]]$$

矩阵写法恰好就是:

$$dW2 = a1^T @ dy_hat$$

Shape 检查:

$$(2,1) = (2,1) @ (1,1)$$

数学直觉: 为什么要转置

$a1$ 原来是行向量 $(1,2)$ 。我们需要得到每个 hidden unit 对每个 output 的 Weight 梯度矩阵 $(2,1)$, 所以把 $a1$ 转成列向量 $(2,1)$ 再与 dy_hat 相乘。

八、【Backward 3】db2 与 da1

Bias 直接加到输出上，所以：

$$db2 = dy_hat = 3$$

接着必须把梯度继续传回 a1。因为：

$$y_hat = a1 @ W2 + b2$$

每个 a1_i 对 y_hat 的局部导数就是对应的 W2_i，因此：

$$da1 = dy_hat @ W2^T$$

本例：

$$da1 = [3] @ [2, -1] = [6, -3]$$

Shape：

$$(1, 2) = (1, 1) @ (1, 2)$$

核心概念：最重要的直觉

Weight 在 Forward 中把 activation 推向下一层；同一套 Weight 的转置，在 Backward 中把后方梯度重新分配回前一层各维度。

九、【Backward 4】穿过 ReLU: $da1 \rightarrow dz1$

Forward 时:

$$z1=[3,-2] \rightarrow a1=[3,0]$$

ReLU mask:

$$\text{ReLU_prime}(z1) = [1,0]$$

所以:

$$dz1 = da1 * \text{ReLU_prime}(z1)$$

$$dz1 = [6,-3] * [1,0] = [6,0]$$

常见误区 / 风险: 第二个 hidden unit 发生了什么

第二个隐藏神经元虽然从第二层收到了 $da1_2=-3$, 但它在 Forward 时 $z1_2=-2$, 被 ReLU 关掉; Backward 时乘以0后, 梯度无法继续传到它前面的 $w1$ 第二列。

十、【Backward 5】第一层 Weight: $dW1 = x^T @ dz1$

第一层:

$$z1 = x @ W1 + b1$$

它与第二层 Linear 的规则完全一样，只是维度更大。

$$dW1 = x^T @ dz1$$

本例:

$$x^T = [[1],[2],[-1]], dz1=[6,0]$$

外积得到:

$$dW1 = [[6,0],[12,0],[-6,0]]$$

Shape:

$$(3,2) = (3,1) @ (1,2)$$

W1 位置	梯度	来源
$W1[0,0]$	6	$x1=1 \times dz1_1=6$
$W1[1,0]$	12	$x2=2 \times 6$
$W1[2,0]$	-6	$x3=-1 \times 6$
$W1[:,1]$	全部0	$dz1_2=0$

十一、【Backward 6】db1 与 dx

Bias 梯度:

$$db1 = dz1 = [6,0]$$

要继续把梯度传给更前面的层:

$$dx = dz1 @ W1^T$$

本例:

$$dx = [6,0] @ W1^T = [3,6,-3]$$

Shape:

$$(1,3) = (1,2) @ (2,3)$$

核心概念: 为什么 dx 很重要

在一个真实深层网络里, 这里的 x 往往不是原始数据, 而是上一层输出。所以 dx 会成为上一层的“上游梯度”, Backprop 就这样一层接一层继续。

十二、【公式总表】两层 MLP Backpropagation

Forward	Backward
$z1 = x @ W1 + b1$	$dy_hat = 2(y_hat - target)$
$a1 = ReLU(z1)$	$dW2 = a1^T @ dy_hat$
$y_hat = a1 @ W2 + b2$	$db2 = sum(dy_hat)$
$L = (y_hat - target)^2$	$da1 = dy_hat @ W2^T$
	$dz1 = da1 * ReLU_prime(z1)$
	$dW1 = x^T @ dz1$
	$db1 = sum(dz1)$
	$dx = dz1 @ W1^T$

数学直觉：不要死背

真正需要掌握的是 Linear 的通用三件套： $dW = input^T @ upstream$ ， $db = \text{对 batch 维求和}$ ， $dInput = upstream @ W^T$ 。ReLU 只是在两层 Linear 之间多一个逐元素 gate。

十三、【Shape 自检】为什么这些矩阵乘法都合法？

量	Shape
x	(1,3)
W1 / dW1	(3,2)
z1,a1,da1,dz1	(1,2)
b1 / db1	(2,)
W2 / dW2	(2,1)
y_hat,dy_hat	(1,1)
b2 / db2	(1,)
dx	(1,3)

逐式检查：

$$a1^{T(2,1)} @ dy_hat(1,1) \rightarrow dW2(2,1)$$

$$dy_hat(1,1) @ W2^{T(1,2)} \rightarrow da1(1,2)$$

$$x^{T(3,1)} @ dz1(1,2) \rightarrow dW1(3,2)$$

$$dz1(1,2) @ W1^{T(2,3)} \rightarrow dx(1,3)$$

科研连接：调试习惯

以后看到矩阵梯度公式不确定时，不要先背结论，先问：我需要得到什么 Shape？哪些已有矩阵乘起来能得到这个 Shape，同时语义也匹配？

十四、【为什么 Backward 经常出现 Transpose?】

Forward 中 Linear 做:

$$Y = X @ W$$

要得到 W 的梯度, 需要把“每个输入特征”与“每个输出方向的上游梯度”两两组合, 因此:

$$dW = X^T @ dY$$

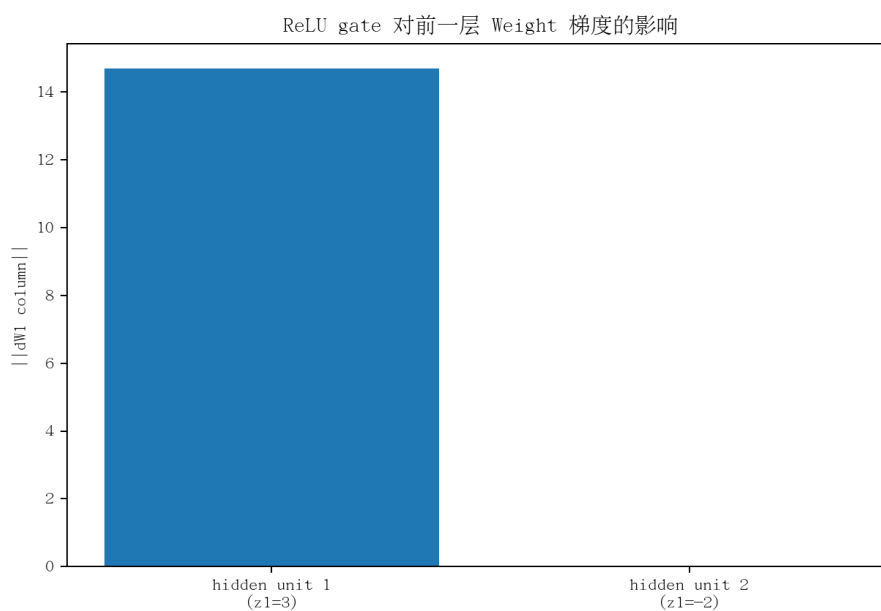
要把输出方向的梯度重新分配回输入特征方向, 则:

$$dX = dY @ W^T$$

数学直觉: Transpose 不是人为补丁

转置来自“索引配对关系”和 Shape。Forward 时 W 的行连接 input feature, 列连接 output feature; Backward 时必须沿这些连接反向传播, 所以自然出现 W^T 。

十五、【图解】一个隐藏神经元被关闭，会影响哪些梯度？



本例第二个 hidden unit:

$$z1_2=-2 \rightarrow \text{ReLU mask}=0 \rightarrow dz1_2=0$$

因此:

$$dW1[:,2] = x^T * 0 = 0$$

并且:

$$db1_2 = 0$$

常见误区 / 风险: 注意边界

这不代表第二列 $W1$ 永远不更新。换一个样本或参数状态, 只要 $z1_2>0$, 它就可能重新收到梯度。这里讨论的是当前 Forward 路径上的局部结果。

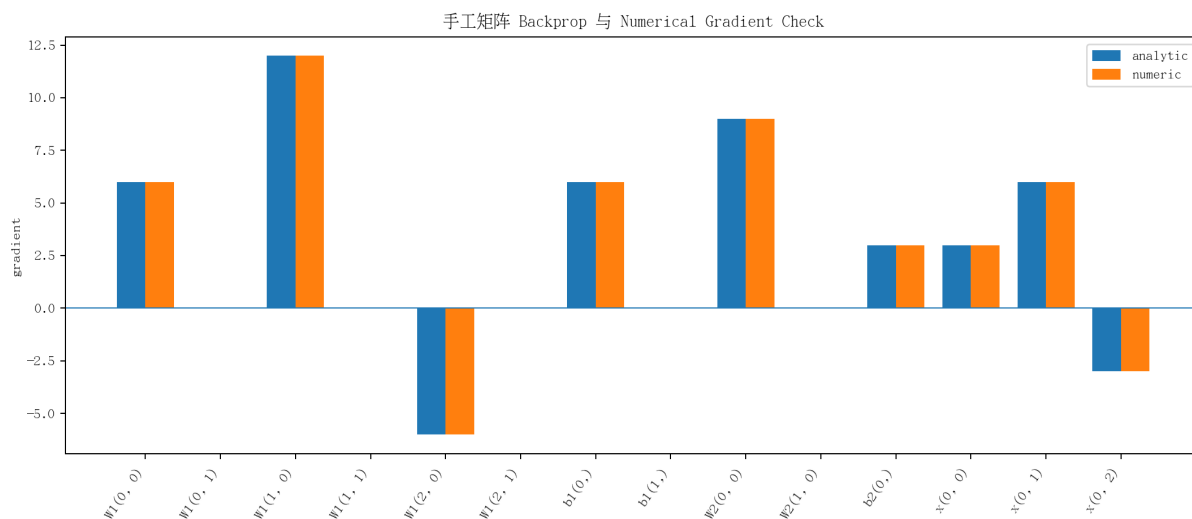
十六、【Numerical Gradient Check】怎样知道矩阵公式没写错？

对每个参数元素 q 单独做 central difference:

$$g_{\text{numeric}} = [L(q+\text{eps}) - L(q-\text{eps})] / (2\text{eps})$$

实验逐元素检查 $W1$ 、 $b1$ 、 $W2$ 、 $b2$ 和 x 。最大误差:

$$\text{max error} \approx 1.9\text{e-}10$$



科研连接：科研用途

自己写自定义层、自定义 Loss 或自定义 backward 时，Gradient Check 是非常有价值的正确性检查；但它太慢，不是训练算法。

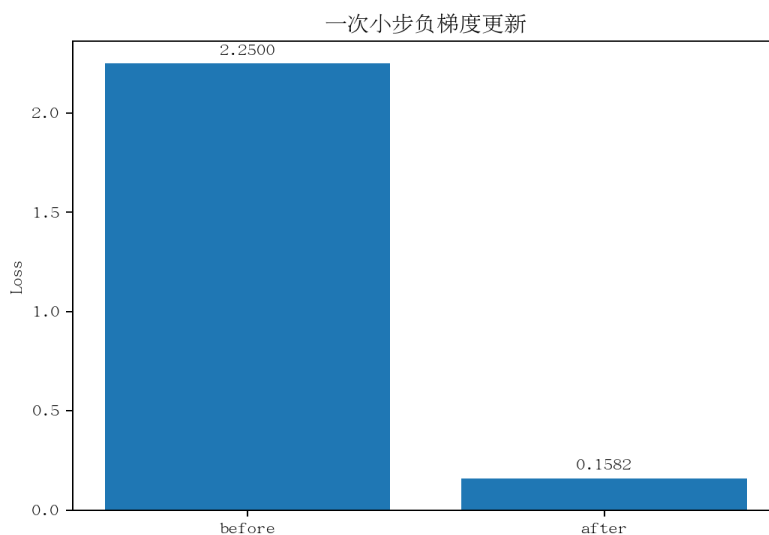
十七、【参数更新】Gradient 算出来之后怎么用？

先用最简单的一步 Gradient Descent:

$$W \leftarrow W - \text{learning_rate} * dW$$

本例同时更新 $W1$ 、 $b1$ 、 $W2$ 、 $b2$, $\text{learning rate}=0.01$:

Loss: 2.25 -> 0.158245



常见误区 / 风险：不要过度解读单步下降

这里只证明当前局部小步方向有效。真正训练时 Loss 会受 Batch、非凸地形、学习率、Optimizer、Normalization 等共同影响。

十八、【Batch 版本】矩阵 Backprop 为什么特别重要？

真实训练不会一次只算一个样本。若：

$$X: (B, D_{in}), W1: (D_{in}, H), W2: (H, D_{out})$$

那么：

$$Z1 = X @ W1 + b1 \rightarrow (B, H)$$

$$Y_hat = A1 @ W2 + b2 \rightarrow (B, D_{out})$$

若 Loss 对 batch 求 mean，则输出梯度包含 $1/B$ 。通用矩阵公式仍然不变：

$$dW2 = A1^T @ dY_hat$$

$$dW1 = X^T @ dZ1$$

$$db = \text{sum over batch dimension}$$

科研连接：Reduction 会改变梯度尺度

mean 与 sum 不只是 Loss 数字不同，它们会直接改变梯度整体尺度。以后比较论文实现时，必须确认 loss reduction、batch size 和 gradient accumulation。

十九、【PyTorch Autograd 对照】

本讲代码对完全相同的参数运行：

```
z1 = x @ W1 + b1
a1 = torch.relu(z1)
y_hat = a1 @ W2 + b2
loss = torch.sum((y_hat - target) ** 2)
loss.backward()
```

PyTorch 得到：

梯度	手工 / NumPy	PyTorch
dW2	[[9],[0]]	完全一致
db2	[3]	完全一致
dW1	[[6,0],[12,0],[-6,0]]	完全一致
db1	[6,0]	完全一致
dx	[3,6,-3]	完全一致

max NumPy-vs-PyTorch diff = 0

核心概念：现在怎样理解 Autograd

PyTorch 不是跳过了这些数学，而是在计算图上自动保存 Forward 所需中间量，然后按逆序高效执行等价的矩阵 backward。

二十、【与 Transformer / LLM 的关系】

组件	与本讲关系
Transformer FFN	本质就是更宽的 Linear -> GELU/SiLU -> Linear; Backprop 结构与本讲同类
Q/K/V Projection	都是 $X @ W$; 每个投影矩阵最终都有 $dW = X^T @ \text{upstream}$ 的结构
Output Projection	Attention 后的 Linear 同样按矩阵规则反传
Embedding 之后的 hidden states	其梯度会沿每层 Transformer Block 一直向前传播
LLM Pretraining Loss	Cross Entropy 在末端产生梯度, 经过很多矩阵层、Activation、Residual、Norm 逐层传播

科研连接: 以前 LLM 课里的“反向传播”

以前看到的是宏观流程: Loss -> Backprop -> 参数更新。现在你已经能看懂其中最常见的矩阵层到底怎样产生参数梯度、怎样把梯度继续传到更前面。

二十一、【常见误区】

常见误区 / 风险：误区1: dW_2 是 W_2 乘某个梯度

不是。 dW_2 来自输入 activation 与输出上游梯度的外积: $A^T \otimes dY$ 。

常见误区 / 风险：误区2: Transpose 是为了让代码不报错才加的

错误。转置来自连接关系和索引配对，只是 Shape 同时帮助我们验证公式。

常见误区 / 风险：误区3: ReLU 后某个单元为0，后层 Weight 梯度就一定全为0

只对与该 activation 相关的那部分梯度为0；其他 hidden unit 仍可正常贡献。

常见误区 / 风险：误区4: db 就等于 upstream 在任何情况下都完全一样

单样本标量例子如此；Batch 情况通常需要沿 batch 维求和，具体还受 loss reduction 影响。

常见误区 / 风险：误区5: 梯度矩阵里的大数说明某个参数“最重要”

梯度是当前参数、当前样本/Batch 附近的一阶局部敏感度，不等于稳定的重要性或因果贡献。

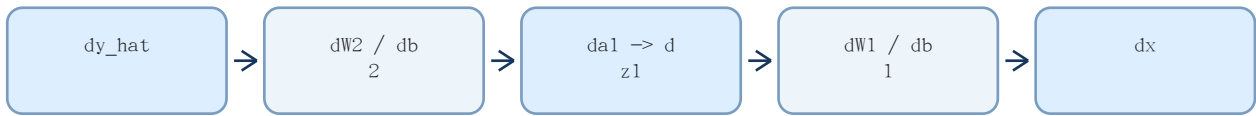
常见误区 / 风险：误区6: 手工推完两层 MLP 就需要手推整个 Transformer

不需要。目的只是建立底层心智模型；之后框架会自动求导，你需要能解释和诊断，而不是手推数十亿参数。

二十二、【代码实验】本讲做什么？

实验	问题	结果
1. 完整 Forward	两层 MLP 每一步数值是什么	$z1=[3,-2]$, $a1=[3,0]$, $y_hat=6.5$, $L=2.25$
2. 手工矩阵 Backward	所有梯度怎么算	$dW2$ 、 $dW1$ 、 db 、 dx 全部得到
3. Shape Check	Gradient Shape 是否与参数匹配	全部匹配
4. Gradient Check	矩阵公式是否正确	最大误差约 $1.9e-10$
5. ReLU gate	关闭单元怎样影响前层	$W1$ 对应整列梯度为0
6. 一步更新	负梯度小步是否降 Loss	$2.25 \rightarrow 0.158245$
7. Batch 预览	共享参数怎样聚合梯度	矩阵公式不变，沿 batch 聚合
8. PyTorch 对照	Autograd 是否等价	最大差异0

二十三、本讲知识闭环



到这里，你已经第一次完整看懂了“多层神经网络为什么可以逐层反传梯度”。

核心概念：下一讲

第26讲：NumPy 手写最小 Backprop + Training Loop。我们会把 Forward、Loss、Backward、参数更新放进循环，让一个两层网络真的通过多步训练把 Loss 降下来。